

Lab 9

Working with Stacks and Strings

Prof. Charan Bhaskar

Stack

Stack is an area of memory for keeping temporary data. Stack is used by CALL instruction to keep return address for procedure, RET instruction gets this value from the stack and returns to that offset. Quite the same thing happens when INT instruction calls an interrupt, it stores in stack flag register, code segment and offset. IRET instruction is used to return from interrupt call.

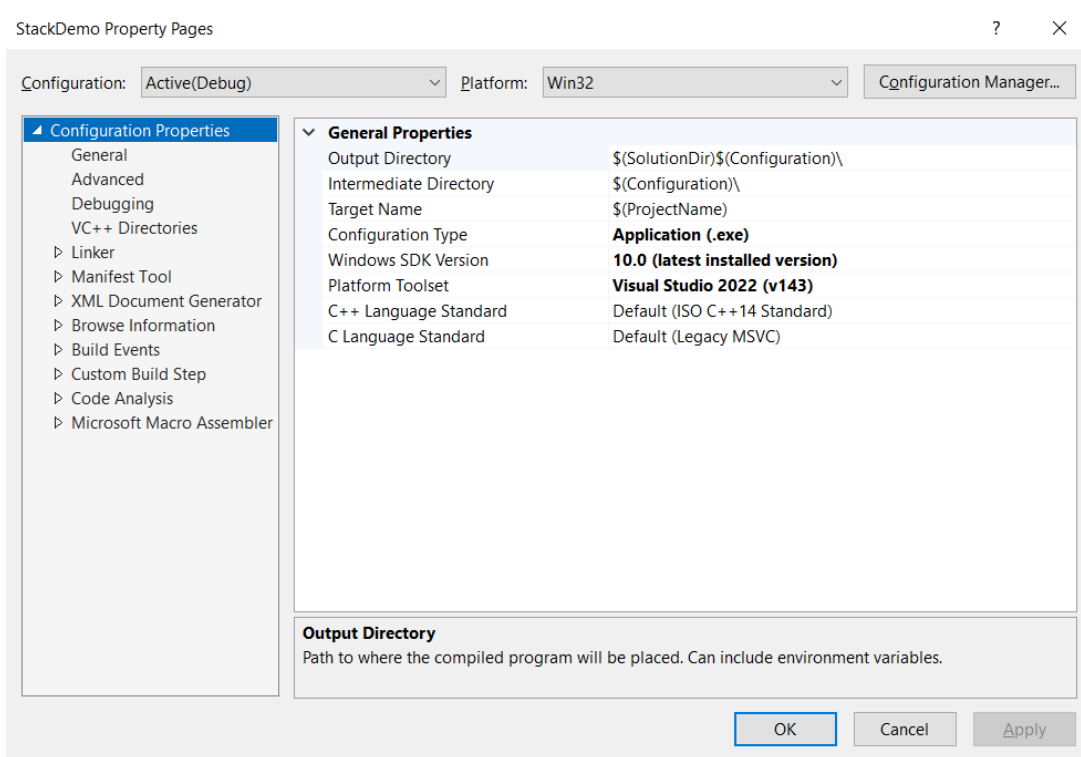
We can also use the stack to keep any other data.

Some useful Stack Instructions:

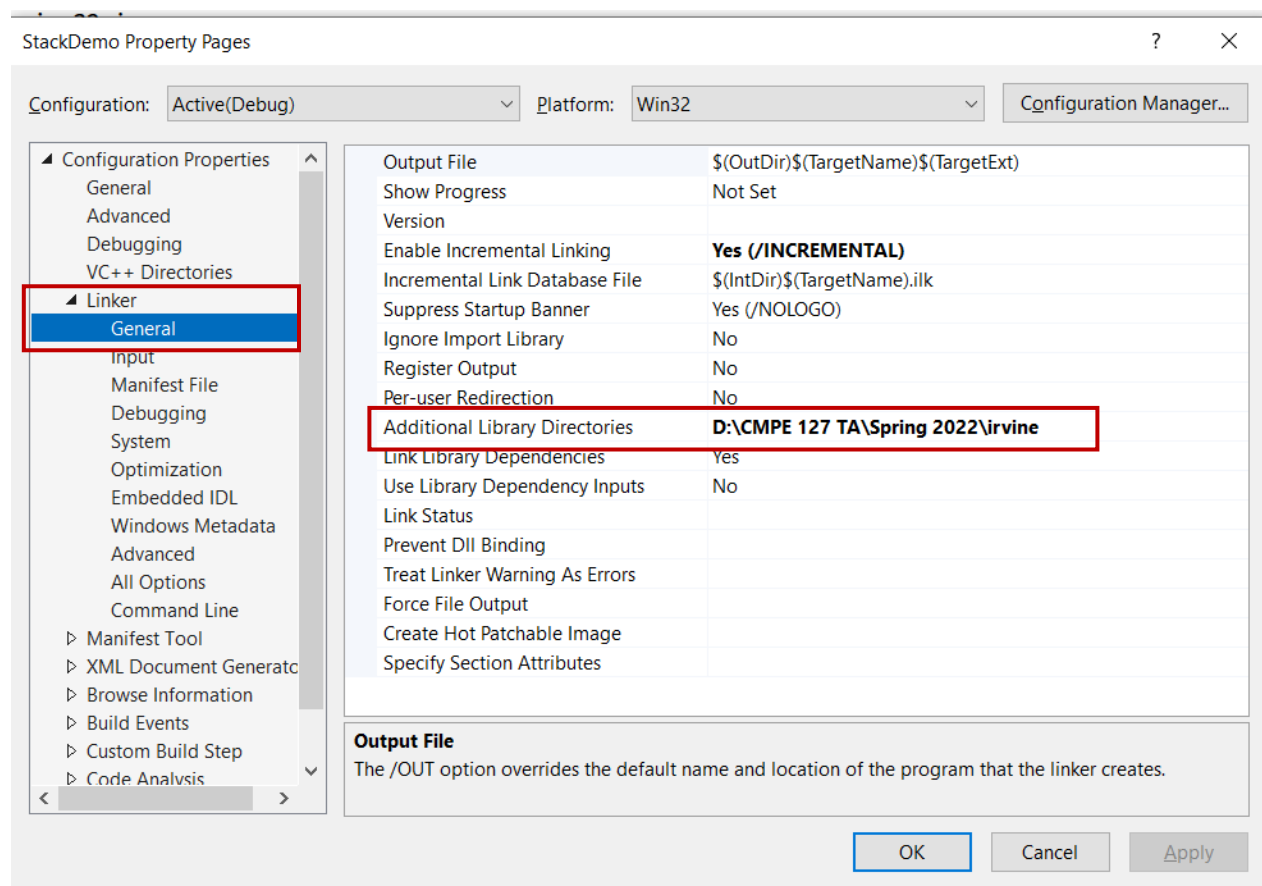
1. **PUSH** and **POP**: To push and pop 16 bit registers argument given.
2. **PUSHFD** and **POPFD**: push and pop the EFLAGS register.
3. **PUSHAD** pushes the 32-bit general-purpose registers on the stack in order: EAX, ECX, EDX, EBX, ESP, EBP, ESI, EDI
4. **POPAD** pops the same registers off the stack in reverse order.
5. **PUSHA** and **POPA**: do the same for 16-bit registers.

Setup Irvine

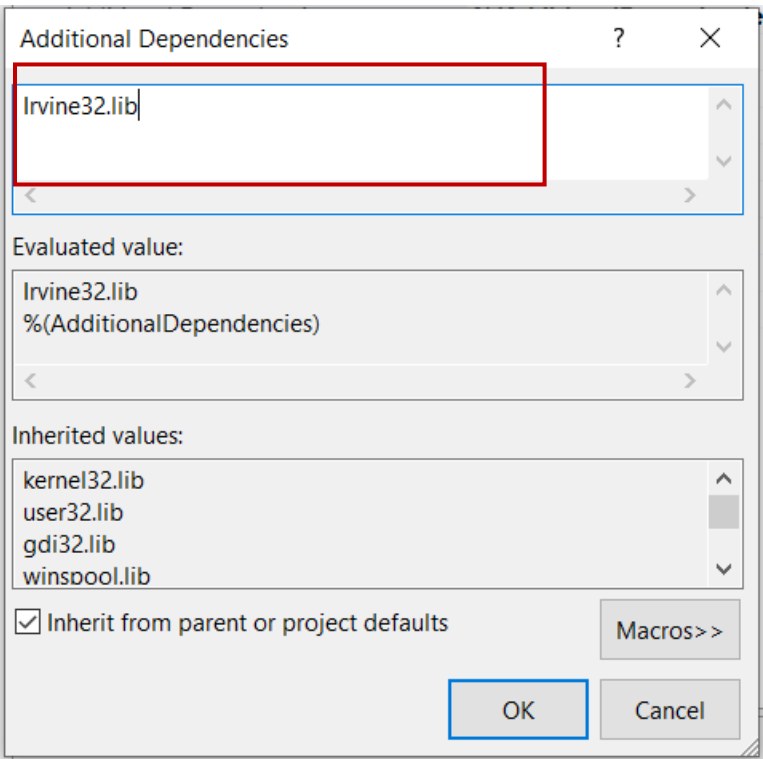
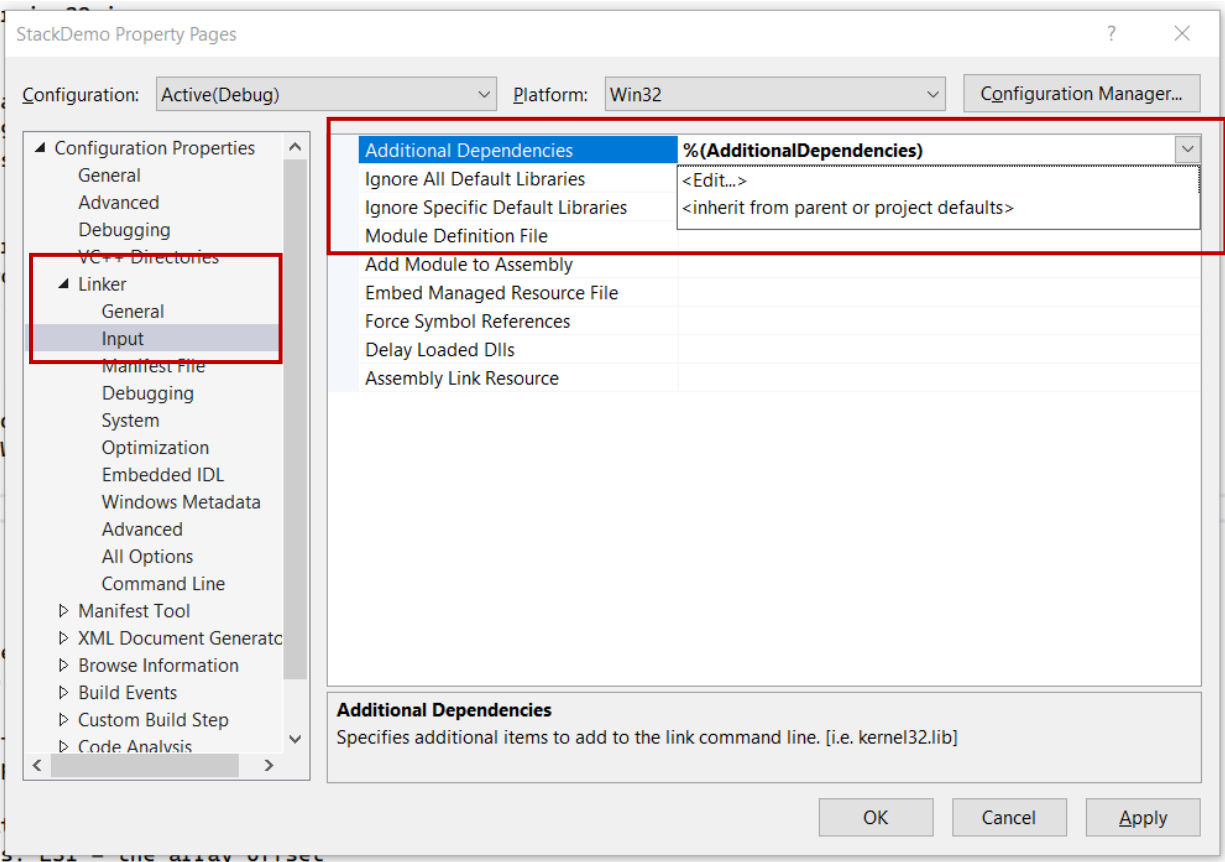
1. Download Irvine zipfile from Canvas → Modules → Lab 9 → Irvine.zip
2. Right click on your project name in solution explorer and open properties.



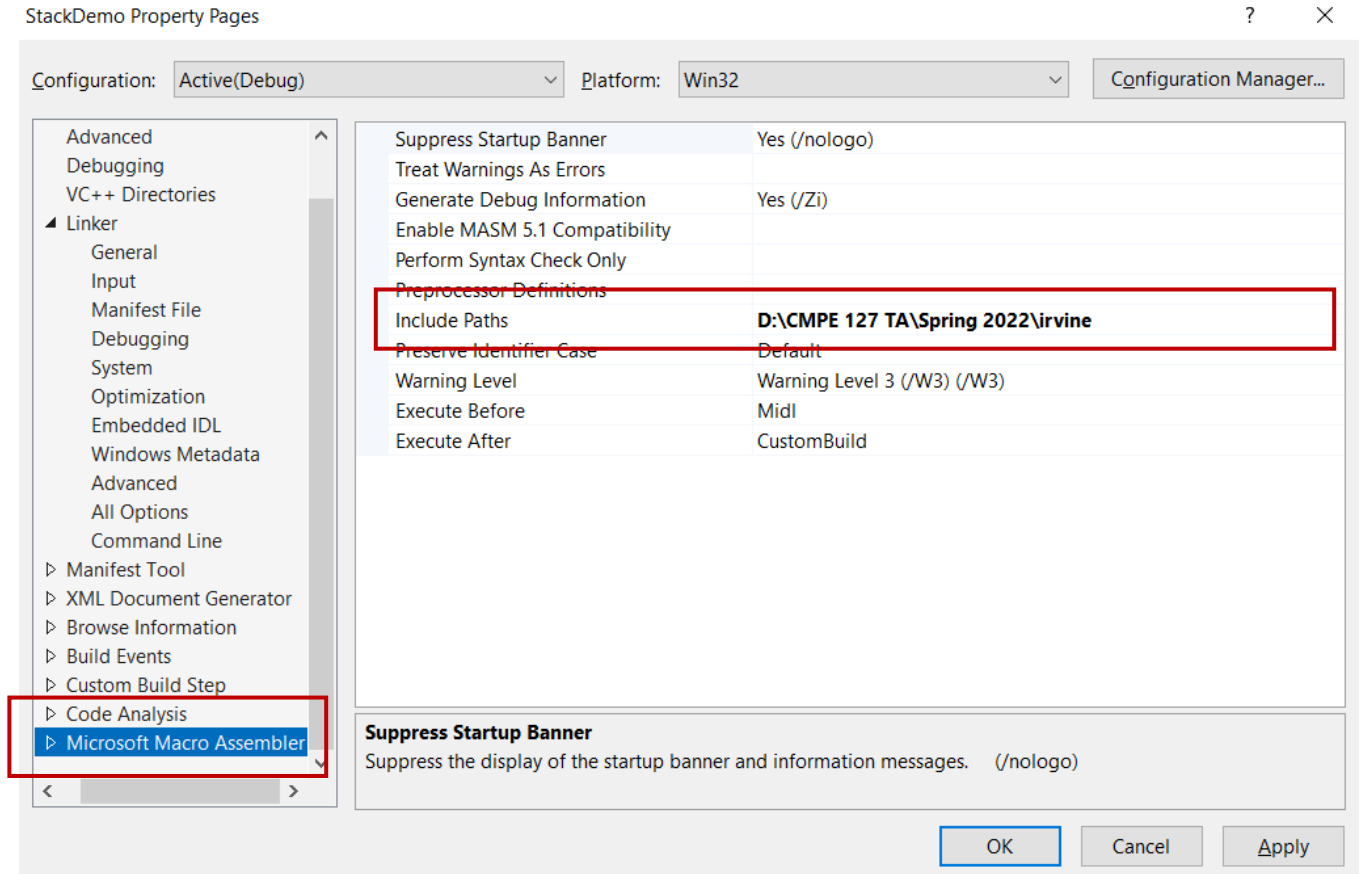
- Go to Linker tab and in General, Go to Additional Library Dependencies and add path to your extracted folder Irvine.



4. Go to Input tab, In Additional Dependencies Go to Edit and add Irvine32.lib as following:



- Go to Microsoft Macro Assembler tab and add the same path in "Include Paths" column.



- Write include Irvine32.inc to use the functions in your code.

```
include Irvine32.inc
```

```
.386
.model flat,stdcall
.stack 4096
ExitProcess PROTO, dwExitCode:dword
```

Sample Program

Program to reverse a string using stack.

```
.data
mymsg byte "Hello World"
reversemsg byte lengthof mymsg DUP (0) ;initialize array as same size of mymsg

.code
main proc
    mov edx, offset mymsg      ;give offset for calling WriteString
    call WriteString
```

```

    mov ebx, offset mymsg      ;initialize base variables for push loop
    mov esi, 0
    mov ecx, lengthof mymsg

push_next:
    mov eax, 0
    mov al, [ebx + esi]
    push ax                    ;push ax to the stack
    inc esi
    loop push_next

    mov ebx, offset reversemsg ;initialize base variables for pop loop
    mov ecx, lengthof mymsg
    mov esi, 0
pop_next:
    pop ax                     ;pop top of stack into ax
    mov [ebx + esi], ax
    inc esi
    loop pop_next

    mov edx, offset reversemsg
    call WriteString

    invoke ExitProcess,0
main endp

```

Lab Work

- Take reference from the above sample program and implement following programs.
- Attach properly commented code in the lab report.
- Explain what your program is doing in detail.
- Add summary and conclusion for the lab.

Question 1:

Extend the sample program to check whether two strings are palindrome or not.

Change mymsg to "level" and check.

Question 2:

Write a program that will print using Irvine perimeter of a circle given a radius.

```

.data
radius WORD 5

```